Project 8

Where Everything Came From | CIS 4720 Database Management and SQL

Points	Time	Work in	Turn in
10	About 2 hours	project8_starter.sql	Your GitHub repository URL

Reference: https://www.cruzjohal.com/database/chapter-5-lab-guide

The due date is listed on Canvas LMS.

What you are building

Your INVENTORY table says what is in the cupboard. It does not say where any of it came from. If a mouse turns out to be faulty, nothing in your database tells you who to ring.

So you add a second table for the suppliers, connect the two with a key, and then ask the question that needed both of them. That is the whole project. One new table, one link, two queries, one view and one account.

Part	What you do	Steps
1	Get set up and run your starter file	1 to 4
2	Build SUPPLIER, link it to INVENTORY, and record who supplied what	5 to 8
3	Join the two tables, then find the supplier you have never bought from	9 to 10
4	Save the join as a view and let a new account read it	11 to 12
5	Publish to a new GitHub repository	13 to 14

> **You write the SQL for Steps 5 through 11**
>
> Same as Lab 8. You get the specification, the data and the result you should end up with, and you decide what the statements look like. Step 12 is the exception, because you have seen CREATE USER only once and have never seen GRANT at all, so both are printed for you.
>
> Start with the course lab guide, which works through every one of these statements:
>
> **https://www.cruzjohal.com/database/chapter-5-lab-guide**
>
> Your textbook, your Lab 8 file and your own notes are all fair game too. Looking syntax up is what working developers do all day. Copying a classmate's file is not, and two identical submissions are easy to spot.

Step	What you build	Textbook	You have done this before in
5	The SUPPLIER table	5.3	Lab 8 Step 6
6	Three rows of data	5.5	Lab 8 Step 7
7	A new column and a foreign key on INVENTORY	5.4	New, but the lab guide covers it under 5.4
8	An update that fills the new column in	5.5	Lab 8 Step 8
9	A join across the two tables	4.9	New, and the lab guide covers it under joins
10	An outer join to find what is missing	4.9	New, and the lab guide covers it too
11	A view built on the join	5.6	Lab 8 Step 9
12	An account and a grant	Not covered	Printed for you in Step 12

Five of the eight you have already done once in Lab 8, with different table and column names. Only Steps 7, 9 and 10 are genuinely new, and the lab guide works through all three.

> **Label and comment every step**
>
> Each step gets a heading comment carrying its number, and every statement gets a short comment above it in your own words. The eight headings are already written into project8_starter.sql near the bottom, so write your statements underneath each one.
>
> It is worth a full point. Without the headings, work you did correctly cannot be given credit for the step it belongs to.

Part 1. Get set up

The lab PCs are wiped every night, so the database from Lab 8 is gone. The starter file puts it back, which is the whole reason you keep files.

Step 1. Make a folder on OneDrive and get the starter file into it

Through the OneDrive app on the PC, not the browser. A folder made in the browser does not always appear on the PC straight away. Create:

```
CIS4720_Project8
```

Then download **project8_starter.sql** from Canvas and save it into that folder. This file already contains everything Lab 8 asked for, so it does not matter how Lab 8 went for you. Check for the cloud icon or green tick on the folder.

Step 2. Create the schema at the command prompt

Do this before you open VS Code. SQLTools connects to a schema that already exists but will not create one. Open the Windows command prompt and start the MySQL client:

```
mysql -u root -p
```

Type the password your instructor gave you. Nothing appears while you type. At the **mysql>** prompt, create a schema called **inventory_db**, then leave. You wrote both of those statements in Lab 8 Step 2.

Leave this window open. You come back to it in Step 12.

Step 3. Open the folder in VS Code and connect

File, then **Open Folder**, then CIS4720_Project8. Open the folder, not the file, so SQLTools has somewhere to save the connection. You should still have the SQLTools and SQLTools MySQL extensions from Lab 8, and the GitHub Pull Requests and Issues extension you need in Part 5. Check the Extensions panel if you are not sure.

Add a connection to **inventory_db** on localhost port 3306 as root, the same as last time. Test it, then save it.

Step 4. Run the starter file

Open project8_starter.sql, right click in it and choose **Run This File**. Then confirm it worked:

Check	What you should see
Rows in INVENTORY	3
The keyboard's unit_cost	59.99, because Lab 8 dropped its price
SHOW FULL TABLES	INVENTORY and INVENTORY_COPY as BASE TABLE, INVENTORY_VALUE as a VIEW

If any of that is missing, stop and sort it out. Everything after this depends on those three rows.

> **Then comment out the create and insert lines**
>
> Run the file a second time and every CREATE and every INSERT fails, because the tables already exist and the rows are already there. The errors are harmless but they bury the results you want to read.
>
> Select Sections 2, 3 and 4 and press **Ctrl + /**. Leave the SELECT statements in Section 6 alone, because reading is always safe to repeat. Do not use /* and */ for this, because block comments do not nest and the file already has comments in it.
>
> From here on, run one statement at a time instead. Put the cursor on it and press **Ctrl + E** twice.

Part 2. Build SUPPLIER and connect it

Four steps, and three of them are things you did in Lab 8 with different names.

Step 5. Create the SUPPLIER table

Create a table named **SUPPLIER** with these four columns, in this order:

Column	Type	Rules
supplier_id	INT	Never empty. This is the primary key
supplier_name	VARCHAR(40)	Never empty
city	VARCHAR(30)	May be left empty
phone	VARCHAR(20)	May be left empty. Text, not a number, because of the dashes

This is the same shape as INVENTORY in Lab 8 Step 6, with four columns instead of six. Spell the names exactly as shown, because the grading script looks for them.

When it has run, list the table's columns and check that supplier_id shows **PRI** in the Key column.

Step 6. Add the three suppliers

Insert exactly these three rows, then select everything back:

supplier_id	supplier_name	city	phone
10	Valley Components	Modesto	209-555-0142
11	Delta Office Supply	Stockton	209-555-0177
12	Sierra Tech Parts	Fresno	559-555-0103

Three rows should come back. The phone numbers go in quotes, the same as any other text.

Step 7. Add supplier_id to INVENTORY and link the two tables

INVENTORY has no idea the suppliers exist. Two statements fix that, and the order matters, because a column has to exist before you can put a rule on it.

1. Alter INVENTORY to add a new column called **supplier_id** of type **INT**. Do not make it required. Every row already in the table will get NULL in it, which is correct for now.
2. Alter INVENTORY again to add a **foreign key** on that column pointing at supplier_id in SUPPLIER. Name the constraint **fk_inventory_supplier**.

> **What a foreign key is actually doing**
>
> It tells the server that supplier_id in INVENTORY has to match a supplier_id that really exists in SUPPLIER. From now on the server checks that on every insert and every update, so your data cannot end up naming a supplier that was never entered.
>
> An empty column is allowed. NULL means this row points at nothing yet, which is different from pointing at something that does not exist, and only the second one is refused.

Then list INVENTORY's columns and check supplier_id is on the end.

Step 8. Record which supplier each item came from

Update INVENTORY so the supplier_id column holds these values:

item_id	item_name	supplier_id	which is
1	Laser Mouse	10	Valley Components
2	Mechanical Keyboard	10	Valley Components
3	USB Hub	11	Delta Office Supply

Two items share a supplier, so you can do this in two statements rather than three if you want. Either way is fine. Use a WHERE clause on every one of them, and select the table afterwards to check the right rows changed.

Nothing is assigned to supplier 12. That is deliberate and Step 10 depends on it, so do not fill it in.

> **Prove the key is real**
>
> Try one more update that sets an item's supplier_id to **99**, which is not a supplier you entered. It should be refused with **ERROR 1452, cannot add or update a child row**.
>
> That statement is **supposed** to fail. Write a comment above it saying so, and comment it out once you have seen the error, so your file still runs cleanly.

Part 3. Ask the question that needed both tables

Step 9. Join the two tables

A join brings rows from two tables together by matching a column they share. Here the shared column is supplier_id: it is the primary key in SUPPLIER and the foreign key in INVENTORY, which is exactly the pairing a join is built on.

Write a single SELECT that returns these four columns, sorted by supplier name and then by item name:

item_name	quantity	supplier_name	city
USB Hub	40	Delta Office Supply	Stockton
Laser Mouse	25	Valley Components	Modesto
Mechanical Keyboard	12	Valley Components	Modesto

Three rows, because three items have a supplier. The first two columns come from INVENTORY and the last two from SUPPLIER, and the join is what lets one statement reach both.

> **A tip that makes joins much easier to write**
>
> Give each table a short nickname right after you name it, then use that nickname in front of every column. Once a query touches two tables it is the thing that keeps it readable, and it is how every example in the lab guide is written.
>
> If the server complains that a column name is ambiguous, that is exactly what it is asking for: both tables have a column of that name and you have not said which one you mean.

Step 10. Find the supplier you have bought nothing from

Run your Step 9 query again and look at the supplier names. Sierra Tech Parts is not there. They are in your SUPPLIER table, but no item points at them, and a plain join only returns rows that match on both sides. The supplier quietly vanishes from the report.

An **outer join** is the fix. It keeps every row from one side whether or not it has a match on the other, and fills the missing columns with NULL. Write a query that returns the name and city of every supplier that has no items, which should give you exactly one row:

supplier_name	city
Sierra Tech Parts	Fresno

Two things to work out. Which table goes on the left, so that suppliers are the side being kept. And how to ask for only the rows where the match failed, which means testing a column from INVENTORY for NULL. The lab guide shows this exact pattern.

> **Why this one matters more than it looks**
>
> A plain join would have told you this database has two suppliers. It has three. The report was not wrong about the rows it showed, it was wrong about the ones it left out, and nothing on screen said so.
>
> That is the kind of mistake outer joins exist to prevent, and it is why this step is worth as much as the join itself.

Part 4. Save it and share it carefully

Step 11. Save the join as a view

Your Step 9 query is worth keeping. Create a view named **ITEM_SOURCE** that stores it, returning the same four columns: item_name, quantity, supplier_name and city.

A view is a stored SELECT, so whatever select you got working in Step 9 can be stored as it is. You did this in Lab 8 Step 9, and the only new part is that the select behind it has a join in it this time.

Then select everything from the view. The same three rows should come back, with no join written anywhere in the statement.

Step 12. Create a buyer account and grant it the view

A purchasing clerk needs to see what you buy and from whom. They have no business seeing what anything cost. The view already leaves unit_cost out, so you can hand them the view and withhold the tables.

This is the step where the syntax is printed for you. Type all three statements into your file and run them.

```
DROP USER IF EXISTS 'buyer'@'localhost';

CREATE USER 'buyer'@'localhost' IDENTIFIED BY 'Buyer#2026';

GRANT SELECT ON inventory_db.ITEM_SOURCE TO 'buyer'@'localhost';
```

GRANT SELECT means may read. **ON inventory_db.ITEM_SOURCE** names the one object they may read it from. **TO 'buyer'@'localhost'** names the account. That is the whole statement: a permission, a thing, and a person.

Now prove it. Go to your command prompt window and log in as the new account:

```
mysql -u buyer -p
```

The password is **Buyer#2026**. Then run these three and record what each one gives you:

```
SELECT * FROM inventory_db.ITEM_SOURCE;
SELECT * FROM inventory_db.INVENTORY;
SELECT * FROM inventory_db.SUPPLIER;
```

What comes back is the point of the whole step:

Statement	Result
SELECT from ITEM_SOURCE	Works. Three rows, with no prices in them
SELECT from INVENTORY	ERROR 1142, SELECT command denied to user 'buyer'@'localhost'
SELECT from SUPPLIER	ERROR 1142, denied the same way

The buyer can see every item, how many there are and who supplies them, and cannot see a single price. You never had to hide anything by hand. The view decided which columns exist and the grant decided who could reach them. That is what views are actually for.

Type **exit** to leave the buyer session and go back to VS Code.

> **That password is a throwaway, and this is why**
>
> Your file goes into a public repository in Part 5, so anything in it can be read by anyone. Use **Buyer#2026** exactly as printed. It belongs to an account on a lab PC that gets wiped tonight.
>
> Outside this classroom the rule has no exceptions: a real password never goes into a file that goes into a repository. Teams keep them in environment variables or a secrets manager, and the repository holds only the name of the setting.

Part 5. Publish to a new GitHub repository

Step 13. Publish the folder

Save project8_starter.sql first. If you are not still signed in to GitHub, click **Accounts** at the bottom of the left bar, choose **Sign in with GitHub**, and click **Authorize** in the browser.

1. Click **Source Control** in the left bar.
2. Click **Publish to GitHub**.
3. Name it **cis4720-project8**. It has to be a name you have not used.
4. Choose **Publish to a public repository**. A private one gives your instructor a 404 page instead of your work.
5. Tick **project8_starter.sql**. Leave the **.vscode** folder unticked, because that is the one holding your root password.

> **Sign out of GitHub before you leave**
>
> Shared PC. Your sign in stays on it until somebody removes it. Click **Accounts**, then your username, then **Sign Out**.

Step 14. Check it on github.com and submit the URL

1. Go to github.com, open your profile, then **Repositories**, then **cis4720-project8**.
2. Click project8_starter.sql and read it on the page. Check all eight of your Part 2 through Part 4 steps are there with your comments above them. What is on this page is exactly what gets graded.
3. Open a private or incognito window and paste the URL in. A **404** means the repository is private, so go to **Settings** and change the visibility to public.
4. Paste the URL into the Project 8 submission box on Canvas.

Submit the URL even if you did not finish. Partial work your instructor can read scores better than finished work nobody can reach.

What to turn in

The URL of your **cis4720-project8** repository, pasted into Canvas. It has to contain:

A file called **project8_starter.sql** with your statements for Steps 5 through 12, under the headings already in the file.

A comment in your own words above every statement you wrote.

No **.vscode** folder, because that one holds your password.

How it is graded

Steps	What is being checked	Points
1 to 4	Folder on OneDrive, inventory_db created, starter file running	0.5
5 to 6	SUPPLIER created with the four columns and the key, three rows in it	2.0
7	The supplier_id column added and the foreign key named and working	1.5
8	All three items assigned, supplier 12 left empty, the key tested	1.0
9	A join returning the four columns for all three items	1.5
10	An outer join that finds Sierra Tech Parts	1.0
11	ITEM_SOURCE created and queried	1.0
12	The account created, the view granted, and both tested as the buyer	0.5
All	Required. A comment in your own words above every statement	1.0
13 to 14	A new public repository with the file in it, URL submitted	0.0

Total 10 points. The due date is listed on Canvas LMS.

If something goes wrong

Most of these are a typo rather than a misunderstanding. If a statement will not come out at all, go to the lab guide at **https://www.cruzjohal.com/database/chapter-5-lab-guide** and find the worked version of the same kind of statement.

What you see	What it means
ERROR 1049, unknown database 'inventory_db'	The schema is not there. Go back to Step 2 and create it at the command prompt.
ERROR 1452, cannot add or update a child row	You used a supplier_id that is not in SUPPLIER. Expected at the end of Step 8.
ERROR 1072, key column doesn't exist in table	You tried to add the foreign key before the column. Do Step 7 in the order given.
ERROR 1052 or a complaint that a column is ambiguous	Both tables have a column of that name and you have not said which one. Put the table nickname in front of it.
ERROR 1054, unknown column	A name is misspelled, or you are asking one table for a column that lives in the other.
ERROR 1064, you have an error in your SQL syntax	A typo. The message points at the first thing it could not read, so the mistake is usually just before that.
ERROR 1050, table already exists	You ran a CREATE a second time. Comment it out, or drop the table first.
ERROR 1142, command denied	You are logged in as the buyer, who may only read the view. Expected in Step 12.
ERROR 1396, operation CREATE USER failed	The account exists from an earlier attempt. The DROP USER IF EXISTS line above it is there to prevent this.
Your join returns no rows at all	Step 8 probably did not run, so supplier_id is still NULL on every item and nothing matches.